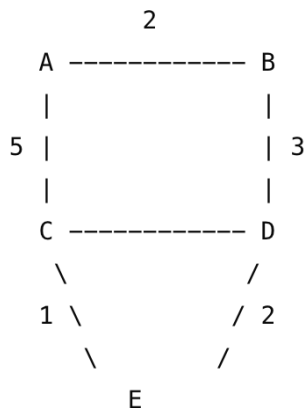


Exercise 2: Link State and Distance Vector Routing

ชื่อ ประดิพัทธ์ ทับทองดี รหัสนิต 6710405036 วันที่ 21/09/2569

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



สมมติฐาน: กราฟเป็นแบบสองทิศทาง และใช้ $\text{cost}(C-D) = 1$ ตามข้อมูลจาก C ในข้อ 4 เนื่องจากรูปไม่ได้ระบุค่านี้

Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

เมื่อ cost เท่ากัน เลือก C ก่อน D; แต่ละแถวแสดงค่าหลังตรวจเพื่อนบ้านของโหนดที่ยืนยันแล้ว

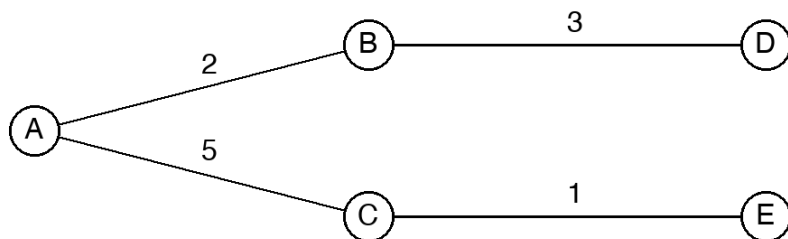
Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	∞ , –	∞ , –
1	B	2, A	5, A	5, B	∞ , –
2	C	2, A	5, A	5, B	6, C
3	D	2, A	5, A	5, B	6, C
4	E	2, A	5, A	5, B	6, C

การอัปเดต: เลือก B $\rightarrow D(D) = \min(\infty, 2+3) = 5$; เลือก C $\rightarrow D(D) = \min(5, 5+1) = 5$ และ $D(E) = \min(\infty, 5+1) = 6$; เลือก D $\rightarrow D(E) = \min(6, 5+2) = 6$; เลือก E $\rightarrow$ ไม่มีค่าลดลง (เส้นย้อนกลับไปโหนดที่ยืนยันแล้วไม่ต้องปรับ)

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:



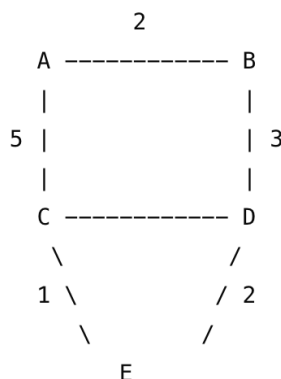
Destination	Shortest Path	Total Cost
B	A → B	2
C	A → C	5
D	A → B → D	5
E	A → C → E	6

Question 3 — Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A → B
C	5	C	A → C
D	5	B	A → B → D
E	6	C	A → C → E

Part 2 — Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 — Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

$D_A(y) = \min\{c(A,B) + D_B(y), c(A,C) + D_C(y)\}$ โดย $c(A,B)=2$ และ $c(A,C)=5$; สำหรับ A เอง $D_A(A)=0$

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	$2 + 0 = 2$	$5 + 4 = 9$	2	B
C	$2 + 4 = 6$	$5 + 0 = 5$	5	C
D	$2 + 3 = 5$	$5 + 1 = 6$	5	B
E	$2 + 5 = 7$	$5 + 1 = 6$	6	C

Question 5 — Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Full Path อนุมานจากกราฟและค่า cost ในโจทย์; Distance Vector ที่ส่งมามีเพียงระยะทาง จึงไม่ได้บอกลำดับโหนดทั้งหมดโดยตรง ค่าที่ใช้เป็นค่าที่ได้รับในรอบนี้ ไม่จำเป็นต้องเป็นค่าหลัง convergence

Destination	Next Hop	Distance	Full Path
B	B	2	A → B
C	C	5	A → C
D	B	5	A → B → D
E	C	6	A → C → E

Part 3 — Routing Convergence

Question 6 — Link Failure

The C–D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.

Your answer should describe:

- Which router(s) detect the failure.
- What information is flooded through the network.
- How routers update their link-state databases.
- How routers calculate new shortest paths.

Distance Vector routing.

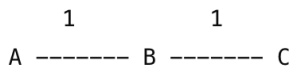
Your answer should describe:

- How routers learn about the failure.
- How updated distance vectors propagate.
- Why routers may temporarily have incorrect routing information.
- How routing eventually converges.

Link State	Distance Vector
C และ D ตรวจพบลิงก์ล่มจากสถานะ interface หรือหมดเวลา Hello	C และ D ตรวจพบโดยตรง ส่วนโหนดอื่นเรียนรู้จาก vector ของเพื่อนบ้าน
C และ D สร้าง LSA ใหม่ ระบุว่า C–D ใช้งานไม่ได้ แล้ว flood ในขอบเขตที่เกี่ยวข้อง	C และ D ตัดลิงก์ C–D ออกจากตัวเลือก แล้วคำนวณระยะทางใหม่ผ่านเพื่อนบ้านที่เหลือ
เราเตอร์รับ LSA รุ่นใหม่และปรับ LSDB ให้ไม่มีลิงก์ C–D	ส่งค่าที่เปลี่ยนให้เพื่อนบ้าน โดย triggered/periodic update ตามโปรโตคอล แล้วส่งต่อเป็นทอด ๆ
แต่ละเราเตอร์รัน Dijkstra ใหม่ แล้วปรับ next hop ในตารางส่งต่อ	ข้อมูลเก่าและการรับ update ไม่พร้อมกันอาจทำให้เลือกเส้นทางผิดหรือเกิด loop ชั่วคราว
เมื่อข้อมูลตรงกันจะได้เส้นทางใหม่; จาก A ไป B,C,D,E ยังมี cost 2,5,5,6 เท่าเดิม	เมื่อข้อมูลใหม่แพร่ครบและเครือข่ายคงที่ ตารางจะนิ่ง เช่น C ไป D เปลี่ยนเป็น C → E → D มี cost 3

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B–C link fails, but B receives stale information from A saying C is reachable at cost 2.

Question 7

1. Why can this create a routing loop?

ตอบ: A ยังส่งแพ็กเก็ตไป C ผ่าน B แต่ B เชื่อข้อมูลเก่าของ A และเลือกไป C ผ่าน A จึงเกิดวงวน $A \rightarrow B \rightarrow A$ เพราะทั้งคู่เข้าใจว่าอีกฝ่ายมีทางไป C

2. Show how the estimated distance to C can increase over successive iterations.

ตอบ: ก่อนล่ม $(D_A(C), D_B(C)) = (2, 1)$; B รับค่าเก่า 2 จาก A จึงได้ $(2, 3)$; A รับ 3 จาก B $\rightarrow (4, 3)$; B รับ 4 $\rightarrow (4, 5)$; A รับ 5 $\rightarrow (6, 5)$; ต่อไปเป็น $(6, 7), (8, 7), \dots$ โดยแต่ละครั้งบวก $\text{cost}(A-B)=1$

3. What is count-to-infinity?

ตอบ: Count-to-infinity คือการที่ค่า metric ของปลายทางที่ไปไม่ถึงเพิ่มขึ้นทีละขั้นจากข้อมูลที่วนกลับมา จนถึงค่า infinity ที่โปรโตคอลกำหนด เช่น RIP ใช้ 16 หมายถึงไปไม่ถึง

4. Why does it occur in Distance Vector routing?

ตอบ: DV แบบพื้นฐานรู้ระยะทางและ next hop ที่เพื่อนบ้านแจ้ง แต่ไม่รู้เส้นทางครบ จึงตรวจไม่ได้ว่าข้อมูลนั้นมีต้นทางจากตนเอง ขาวการขาดเส้นทางจึงแพร่ซ้ำ

5. Name a mitigation approach.

ตอบ: ใช้ split horizon with poisoned reverse เมื่อ A ใช้ B เป็น next hop ไป C จะประกาศค่า $C=\infty$ กลับไปยัง B เพื่อป้องกันวงวนสองโหนดนี้ แต่ไม่รับประกันการแก่วงวนที่มีมากกว่าสองโหนด

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	คำนวณเส้นทางจากกราฟที่รวบรวมได้	คำนวณจากระยะทางที่เพื่อนบ้านแจ้ง
Information maintained	LSDB และตารางเส้นทาง	cost ลิงก์ตรง, vector เพื่อนบ้าน และตารางเส้นทาง
Information exchanged	สถานะและ cost ของลิงก์ใน LSA	ระยะทางหรือ metric ไปแต่ละปลายทาง
Algorithm	Dijkstra (SPF)	Bellman–Ford แบบกระจาย
Topology knowledge	รู้กราฟในพื้นที่หรือ area ของตน	รู้เพื่อนบ้านและระยะทาง ไม่รู้กราฟทั้งหมด
Routing updates	flood เมื่อเปลี่ยนแปลง และ refresh ตามโปรโตคอล	ส่งให้เพื่อนบ้านเป็นระยะ/เมื่อเปลี่ยน ตามโปรโตคอล
Convergence	โดยทั่วไปเร็วหลังตรวจพบและ flood	แบบพื้นฐานอาจช้า โดยเฉพาะเมื่อ cost เพิ่ม
Routing loops	ยังอาจมี microloop ช่วงเปลี่ยนตารางไม่พร้อมกัน	อาจเกิด loop จากข้อมูลเก่าที่ฟังพากัน
Count-to-infinity	ไม่เกิดจากกลไก SPF ของ LS	เกิดได้ใน DV แบบพื้นฐาน
Example protocol	OSPF, IS-IS	RIP

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

เลือก Link State เช่น OSPF แบบหลาย area สำหรับเครือข่าย 500 routers โดยมีเหตุผลดังนี้

1. Convergence โดยทั่วไปเร็ว: เมื่อพบความเปลี่ยนแปลงจะ flood LSA และคำนวณ SPF ใหม่ ช่วยลดช่วงเวลาที่ใช้เส้นทางผิด โดยขึ้นกับ timer และการออกแบบด้วย
2. รู้ topology ภายใน area ทำให้คำนวณเส้นทาง cost ต่ำสุดและใช้เส้นทาง cost เท่ากัน (ECMP) ได้ ไม่เกิด count-to-infinity แบบ DV พื้นฐาน
3. รองรับลำดับชั้น: แบ่ง area เชื่อมกับ backbone Area 0 และสรุปเส้นทางที่ขอบ area เพื่อลดขอบเขต flooding ขนาด LSDB และภาระ SPF โดยไม่ต้องให้ทุกโหนดรู้รายละเอียดทั้ง 500 routers
4. ข้อเสีย: ใช้หน่วยความจำเก็บกราฟและ CPU คำนวณ SPF มากกว่า DV แบบง่าย และต้องวางแผน area/summary อย่างรอบคอบ เพราะการสรุปเส้นทางอาจลดรายละเอียดการเลือกเส้นทาง

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

กำหนด N = จำนวน routers, E = จำนวนลิงก์สองทิศทาง และ $d(v)$ = จำนวนเพื่อนบ้านของ v ; เปรียบเทียบ LS กับ DV แบบพื้นฐาน โดยสมมติหนึ่งปลายทางต่อ router

ข้อมูลที่เก็บ: LS เก็บกราฟ $O(N+E)$ และตารางเส้นทาง $O(N)$ ต่อ router; DV เก็บตารางตนเอง $O(N)$ และถ้าเก็บ vector ครบทุกเพื่อนบ้านจะใช้ $O(N \cdot d(v))$ เพิ่มเติม โดยไม่เก็บกราฟทั้งหมด

การสื่อสาร: LS flood LSA หนึ่งฉบับผ่าน $O(E)$ ลิงก์ แต่ขนาดฉบับขึ้นกับจำนวนลิงก์ที่ประกาศ; การกระจาย topology ทั้งชุดแบบพื้นฐานมีปริมาณข้อมูล $O(E(N+E))$ โดยประมาณ ไม่รวม ACK/retransmission ส่วน DV ส่ง vector ขนาด $O(N)$ ผ่านทุกลิงก์ ใช้ $O(NE)$ รายการต่อรอบเต็ม และหลายรอบจนคงที่

การคำนวณ: LS ใช้ Dijkstra $O((N+E) \log N)$ เมื่อใช้ adjacency list และ binary heap หรือ $O(N^2)$ แบบ array; DV คำนวณหนึ่งรอบที่ v ใช้ $O(N \cdot d(v))$ และทั้งเครือข่าย $O(NE)$ ต่อรอบ จึงต้องแยกต้นทุนต่อรอบออกจากต้นทุนจน convergence

Convergence: LS ขึ้นกับเวลาตรวจพบลิงก์เสีย การ flood และ SPF; DV เมื่อเริ่มใหม่บนกราฟคงที่แบบ synchronous ใช้ไม่เกิน $N-1$ รอบแลกเปลี่ยนเพื่อเรียนรู้เส้นทางอย่างง่าย แต่เมื่อเกิด failure ข้อมูลเก่าอาจทำให้ count-to-infinity จึงไม่ใช่ขอบเขตนี้กับทุกกรณี

Scalability: intra-domain ขนาดใหญ่มักเลือก OSPF/IS-IS เพราะ convergence และการแบ่ง area/level จำกัดการกระจาย topology และรองรับการสรุปเส้นทาง แม้ต้องแลกกับ CPU หน่วยความจำ และความซับซ้อน ทั้งนี้ DV ที่มีกลไกขั้นสูงอาจมีคุณสมบัติดีกว่า DV แบบพื้นฐาน